

Vysoké učení technické v Brně
Fakulta informačních technologií

Filtrující DNS resolver

Technická správa projektu k predmetu ISA

Autor: Jozef Ondrejčka (xondre16)

Akademický rok: 2025/2026

Dátum: 17.11. 2025

Obsah

1	Úvod	3
1.1	Uvedenie do problematiky	3
1.2	Kľúčové vlastnosti implementácie	4
2	Formulácia problému	5
2.1	Ciele projektu	5
2.2	Povinné požiadavky	5
2.3	Voliteľné (odporúčané) požiadavky	6
3	Architektúra a návrh riešenia	7
3.1	Parametre príkazového riadka	7
3.2	Prehľad systémovej architektúry	8
3.3	Modulová architektúra	8
3.3.1	Základné moduly	8
3.3.2	Moduly DNS protokolu	8
3.3.3	Podporné moduly	9
3.4	Tok dát a spracovanie požiadavok	9
4	Detaily implementácie	10
4.1	Zostavenie a štruktúra projektu	10
4.2	Implementácia DNS protokolu	10
4.3	Filtrovaná logika	10
4.4	Súbežnosť a mapovanie ID	11
4.5	Spracovanie chýb a robustnosť	11
5	Testovanie	12
5.1	Manuálne testy	12
5.1.1	Forwardovanie povolených A dotazov	12
5.1.2	Blokovanie domény	13
5.1.3	Blokovanie subdomén	14

5.1.4	Nepodporované typy dotazov (non-A)	15
5.1.5	Podpora IPv6 klientov (dual-stack)	16
5.1.6	Súbežné dotazy od viacerých klientov	17
5.1.7	Robustnosť voči chybným paketom	19
5.1.8	Výkon a stabilita pod záťažou	20
5.1.9	Spracovanie formátu filter súboru	21
5.1.10	Graceful shutdown a správa signálov	23
5.2	Automatizované testy	24
5.2.1	Prehľad testovania	24
5.2.2	Automatické testy (make test)	24
6	Známe obmedzenia	26
6.1	Obmedzenia protokolu	26
6.2	Obmedzenia funkcií	26
6.3	Design rozhodnutia a zdôvodnenia	26
6.3.1	Prečo REFUSED namiesto NXDOMAIN?	26
6.3.2	Prečo poll() namiesto epoll()?	27
6.3.3	Single-threaded architektúra s poll()	27
6.3.4	Automatické čistenie URL schém z filtra	27
7	Záver	28
8	Literatúra a použité zdroje	29

1 Úvod

1.1 Uvedenie do problematiky

DNS (Domain Name System) je kritickou súčasťou internetovej infraštruktúry, ktorá zabezpečuje preklad ľahko zapamätateľných doménových mien (napr. `www.example.com`) na numerické IP adresy potrebné pre sieťovú komunikáciu. Tento distribuovaný hierarchický systém pomenovaní funguje na princípe klient-server architektúry, kde DNS resolvers spracovávajú dotazy od klientov a vracajú požadované informácie.

Filtračné DNS servery predstavujú špecializovanú kategóriu DNS resolverov, ktoré okrem štandardného prekladu domén poskytujú aj funkciu blokovania prístupu k nežiaducim doménam. Takéto servery sa používajú na:

- Blokovanie reklám a sledovacích domén (ad-blocking)
- Ochranu pred malware a phishing doménami
- Implementáciu rodičovskej kontroly
- Vynucovanie firemných bezpečnostných politík

Technické výzvy pri implementácii DNS filtračného servera zahŕňajú:

- Správne parsovanie binárneho DNS protokolu s kompresiou mien (RFC 1035)
- Efektívne vyhľadávanie domén v rozsiahlych blocklist súboroch
- Riešenie kolízií Transaction ID pri súbežných dotazoch od viacerých klientov
- Podpora IPv4 aj IPv6 (dual-stack networking)
- Robustné spracovanie chybných a potenciálne škodlivých paketov

Tento projekt implementuje kompletný DNS proxy/filtračný server v jazyku C++17, ktorý spĺňa všetky požiadavky zadania a poskytuje robustnú, efektívnu a bezpečnú implementáciu DNS filtrovania.

1.2 Kľúčové vlastnosti implementácie

- **Plná podpora DNS protokolu** – implementácia parsovania a generovania DNS správ v súlade s RFC 1035
- **Filtrovanie domén** – blokovanie zadaných domén a všetkých ich poddomén na základe blokovacieho zoznamu
- **Dual-stack IPv4/IPv6** – podpora klientov aj upstream resolverov cez IPv4 aj IPv6
- **Súbežné spracovanie požiadaviek** – využitie I/O multiplexingu pomocou `poll()` pre obsluhu viacerých klientov naraz
- **Zabránenie kolíziám ID** – prepisovanie ID pri preposielaní, čím sa eliminujú konflikty medzi súbežnými klientmi
- **Plná podpora DNS kompresie** – korektné spracovanie ukazovateľov na mená (vrátane dopredných ukazovateľov)
- **Robustné spracovanie chýb** – generovanie správnych DNS chybových odpovedí (FORMERR, NOTIMP, REFUSED)
- **Ochrana pred spoofingom** – overovanie zdrojovej adresy odpovedí od upstream resolvera
- **Korektné ukončenie** – čisté spracovanie signálov SIGINT a SIGTERM (graceful shutdown)

2 Formulácia problému

Cieľom projektu je vytvoriť DNS filtračný proxy server, ktorý bude fungovať ako spoľahlivý a bezpečný sprostredkovateľ medzi lokálnymi DNS klientmi a verejným (upstream) resolverom. Server musí vedieť selektívne blokovať požiadavky na nežiaduce domény a ich poddomény, pričom všetky povolené požiadavky preposiela ďalej a odpovede vracia pôvodnému klientovi.

2.1 Ciele projektu

Hlavné ciele implementácie sú:

1. Naslúchanie na DNS požiadavky od lokálnych klientov na konfigurovateľnom UDP porte
2. Správne parsovanie DNS správ podľa špecifikácie RFC 1035 (vrátane podpory kompresie mien)
3. Kontrola požadovanej domény (QNAME) voči zoznamu blokovaných domén
4. Preposielanie povolených požiadaviek na zadaný upstream DNS resolver
5. Vrátanie odpovedí pôvodným klientom s korektným mapovaním Transaction ID
6. Bezpečné a efektívne spracovanie súbežných požiadaviek od viacerých klientov naraz

2.2 Povinné požiadavky

- Podpora DNS dotazov typu A (IPv4 adresy)
- Konfigurovateľný upstream resolver – adresa môže byť zadaná ako IP (IPv4/IPv6) alebo doménové meno (riešené cez `getaddrinfo`)
- Konfigurovateľný naslúchací port (predvolená hodnota 53)
- Súbor s blokovanými doménami (jeden záznam na riadok, podpora poddomén)
- Plne funkčné parsovanie DNS správ s podporou kompresie ukazovateľov (vrátane dopredných ukazovateľov)
- Prepisovanie Transaction ID pri preposielaní požiadaviek (ochrana pred kolíziami ID)
- Generovanie správnych chybových DNS odpovedí pri:

- nepodporovanom type dotazu (napr. AAAA, MX, TXT, ...) → RCODE = 4 (Not Implemented)
- chybnom formáte správy → RCODE = 1 (FormErr)
- blokovanej doméne → RCODE = 5 (Refused) alebo 3 (NXDomain)

2.3 Voliteľné (odporúčané) požiadavky

- Režim podrobného výpisu (-v – verbose logging)
- Časové limity a odstraňovanie starých záznamov v mape ID (timeout handling)
- Podpora rôznych formátov koncov riadkov vo filtri (LF, CRLF, CR)
- Automatické odstraňovanie schém `http://`, `https://` a koncových lomítok z riadkov filtra
- Korektné ukončenie programu pri signáloch SIGINT a SIGTERM

3 Architektúra a návrh riešenia

3.1 Parametre príkazového riadka

Program podporuje nasledujúce parametre:

Použitie: `dns -s server [-p port] -f filter_file [-t timeout] [-v] [-h]`

Povinné parametre:

- `-s, -server <server>` – IP adresa (IPv4/IPv6) alebo doménové meno upstream DNS resolvera
- `-f, -filter <filter_file>` – cesta k súboru so zoznamom blokováných domén

Voliteľné parametre:

- `-p, -port <port>` – číslo portu pre príjem dotazov (výchozí: 53, rozsah: 1–65535)
- `-t, -timeout <seconds>` – timeout dotazov v sekundách (výchozí: 10, rozsah: 1–3600)
Rozšírenie: Konfigurovateľný časový limit pre čistenie starých záznamov v ID mapper
- `-v, -verbose` – zapnutie režimu podrobného výpisu (loguje sa resolving adresy, načítavanie filtra, spracovávanie dotazov)
- `-h, -help` – zobrazenie nápovedy a ukončenie programu

Príklady použitia:

Základné použitie s neprivilegovaným portom

```
./dns -s 8.8.8.8 -p 5000 -f filter.txt
```

Použitie výchozieho portu 53 (vyžaduje root oprávnenia)

```
sudo ./dns -s 8.8.8.8 -f filter.txt
```

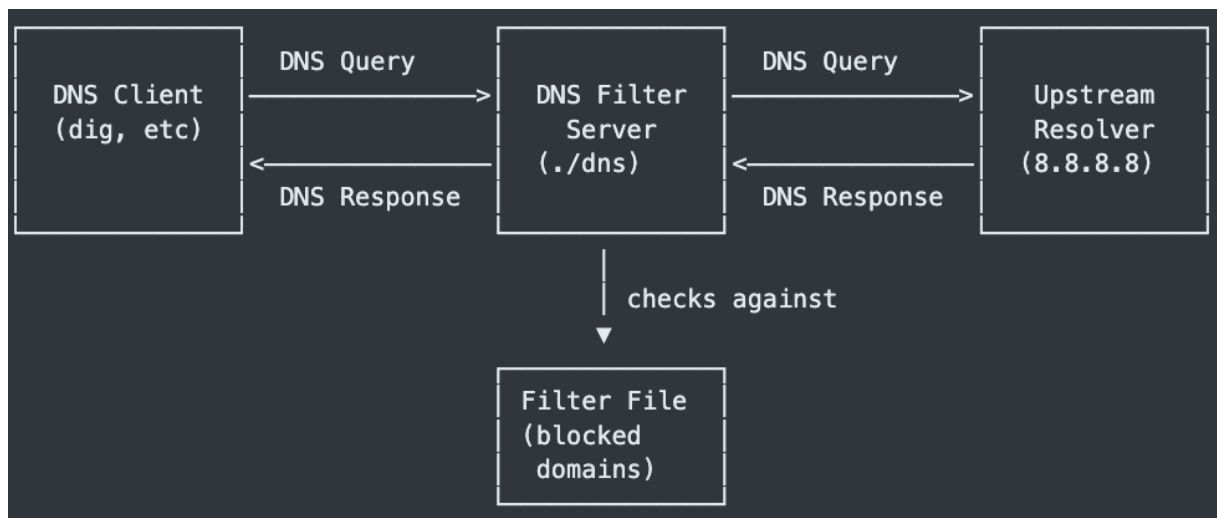
S verbose režimom a vlastným timeoutom

```
./dns -s 8.8.8.8 -p 5000 -f filter.txt -v -t 30
```

IPv6 resolver s dlhými názvami parametrov

```
./dns --server 2001:4860:4860::8888 --port 5353 --filter filter.txt
```


3.2 Prehľad systémovej architektúry



3.3 Modulová architektúra

Projekt je rozdelený do jasne oddelených modulov, čo zvyšuje čitateľnosť, testovateľnosť a údržbu kódu:

3.3.1 Základné moduly

- `main.cpp` – vstupný bod programu, inicializácia a spustenie servera
- `arg_parser` – parsovanie a validácia parametrov príkazového riadka
- `server` – hlavný cyklus, správa socketov a smerovanie požiadaviek
- `query_handler` – rozhodovanie o filtrovaní a ďalšom postupe

3.3.2 Moduly DNS protokolu

- `dns_protocol` – parsovanie hlavičky a základných častí DNS správ
- `dns_compression` – dekompresia a práca s ukazovateľmi v DNS menách (vrátane dopredných)
- `dns_response` – tvorba DNS odpovedí (ako povolených, tak chybových)

3.3.3 Podporné moduly

- `filter` – načítanie a správa zoznamu blokováných domén
- `domain_utils` – normalizácia domén, validácia podľa RFC, kontrola poddomén
- `id_mapper` – mapovanie Transaction ID (orig. ID + klient → nové ID)
- `socket_manager` – vytváranie, bindovanie a správa UDP socketov (IPv4 + IPv6)
- `addr_resolver` – riešenie doménového mena upstream resolvera cez `getaddrinfo()`
- `logger` – centralizované logovanie s podporou verbose režimu

3.4 Tok dát a spracovanie požiadavok

1. Klient odošle DNS dotaz → naslúchací socket (monitorovaný cez `poll()`)
2. Parsovanie DNS hlavičky a sekcie otázky
3. Extrahovanie QNAME, QTYPE a QCLASS
4. Kontrola QTYPE (musí byť A = 1)
5. Porovnanie domény s blokovacím zoznamom
 - Ak je doména blokována → vygenerovať odpoveď s RCODE = 5 (Refused) alebo 3 (NXDomain) → odoslať klientovi
 - Ak je doména povolená → pokračovať
6. Vygenerovanie nového Transaction ID
7. Uloženie mapovania: (zdrojová adresa klienta, port, pôvodné ID) → nové ID
8. Prepis Transaction ID v pakete
9. Preposlanie upraveného dotazu na upstream resolver
10. Čakanie na odpoveď cez `poll()`
11. Overenie zdrojovej adresy odpovede (ochrana pred spoofingom)
12. Vyhľadanie mapovania podľa prichádzajúceho ID
13. Obnovenie pôvodného Transaction ID v odpovedi
14. Odoslanie odpovede späť pôvodnému klientovi
15. Odstránenie záznamu z mapovacej tabuľky

4 Detaily implementácie

Projekt je napísaný v jazyku C++17 s dôrazom na čistotu, prenositeľnosť a robustnosť. Používa sa iba štandardná knižnica C++ a POSIX sokety.

4.1 Zostavenie a štruktúra projektu

- Zostavuje sa cez make s automatickými závislosťami
- Flagy: `-std=c++17 -Wall -Wextra -pedantic -O2`
- Žiadne manuálne `new/delete` – všade RAII (`std::string`, `std::unordered_map`, `std::vector`)
- Valgrind: 0 únikov pamäte

Štruktúra adresárov je jednoduchá:

```
src/      - zdrojové súbory
include/  - hlavičkové súbory
build/    - objektové súbory a závislosti
```

4.2 Implementácia DNS protokolu

- Plne podľa RFC 1035
- Správna konverzia big-endian $\leftrightarrow$ little-endian (`ntohs`, `htons`)
- Podpora kompresie mien (ukazovatele dozadu aj dopredu)
- Detekcia cyklov a kontrola hraníc paketu (ochrana pred chybnými správami)
- Parsovanie len hlavičky a sekcie Question (ostatné časti sa preposielajú bez zmeny)

4.3 Filtračná logika

- Blokovací zoznam sa načítava zo súboru (jeden riadok = jedna doména)
- Podpora komentárov (`#`), prázdnych riadkov a všetkých koncov riadkov (LF, CRLF, CR)
- Automatické odstraňovanie `http://`, `https://` a koncových lomítok

- Domény sa ukladajú do `std::unordered_set<std::string>`
- Kontrola poddomén: ak je v zozname `example.com`, blokujú sa aj `www.example.com`, `sub.www.example.com` atď.
- Validácia domén podľa RFC (povoľuje aj číslice na začiatku labelu)

4.4 Súbežnosť a mapovanie ID

- Hlavná slučka používa `poll()` – monitoruje naslúchací aj upstream socket
- Prepisovanie Transaction ID pri preposielaní (rieši kolízie ID)
- Mapovanie: nové ID $\rightarrow$ (adresa klienta, pôvodné ID)
- Mapa je chránená `std::mutex`, všetko ostatné je single-threaded
- Časový timeout 10 sekúnd + automatické čistenie starých záznamov

4.5 Spracovanie chýb a robustnosť

- Vždy sa odpovedá klientovi – nikdy sa packet len tak nezhodí
- Generované chybové odpovede:
 - FORMERR (1) – chybný formát správy
 - NOTIMP (4) – nepodporovaný typ dotazu (napr. AAAA, MX)
 - REFUSED (5) – blokována doména
- Ochrana pred spoofingom – kontrola zdrojovej adresy odpovede od upstreamu
- Všetky chyby a varovania sa vypisujú na `stderr` v jednotnom formáte
- Program nikdy nespadne (žiadny SEGFAULT ani pri úplne pokazených paketoch)

5 Testovanie

5.1 Manuálne testy

5.1.1 Forwardovanie povolených A dotazov

Cieľ testu: Overiť, že server správne preposiela neblokované dotazy typu A na upstream resolver a vráti klientovi platnú odpoveď s pôvodným Transaction ID.

Príkazy na spustenie:

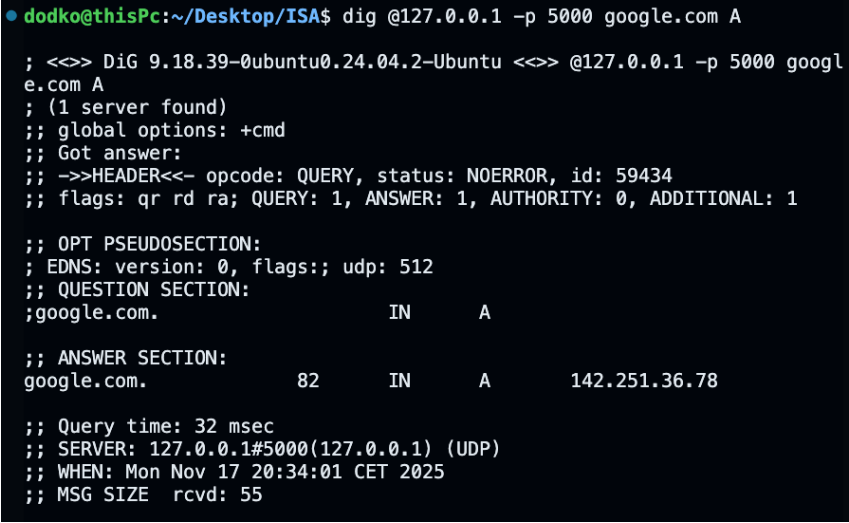
```
# Terminál 1 - spustenie servera
./dns -s 8.8.8.8 -p 5300 -f filter.txt -v
```

```
# Terminál 2 - vykonanie dotazu
dig @127.0.0.1 -p 5300 google.com A
```

Očakávaný výsledok:

- status: NOERROR
- V sekcii ANSWER sa zobrazí aspoň jedna IPv4 adresa (napr. 142.251.129.14)

Výsledok testu:



```
• dodko@thisPc:~/Desktop/ISA$ dig @127.0.0.1 -p 5000 google.com A
; <<> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<> @127.0.0.1 -p 5000 googl
e.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<-- opcode: QUERY, status: NOERROR, id: 59434
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;google.com.                IN      A

;; ANSWER SECTION:
google.com.                 82      IN      A      142.251.36.78

;; Query time: 32 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)
;; WHEN: Mon Nov 17 20:34:01 CET 2025
;; MSG SIZE rcvd: 55
```

Obrázek 1: Úspešné forwardovanie dotazu na google.com

PASSED – Server korektne preposlal dotaz, zachoval kompresiu v odpovedi a vrátil odpoveď pôvodnému klientovi.

5.1.2 Blokovanie domény

Cieľ testu: Overiť, že server okamžite blokuje doménu uvedenú vo filter súbore a vráti chybovú odpoveď bez kontaktovania upstream resolvera.

Príkazy na spustenie:

```
# Overenie, že je doména vo filtri
grep "doubleclick.net" filter.txt

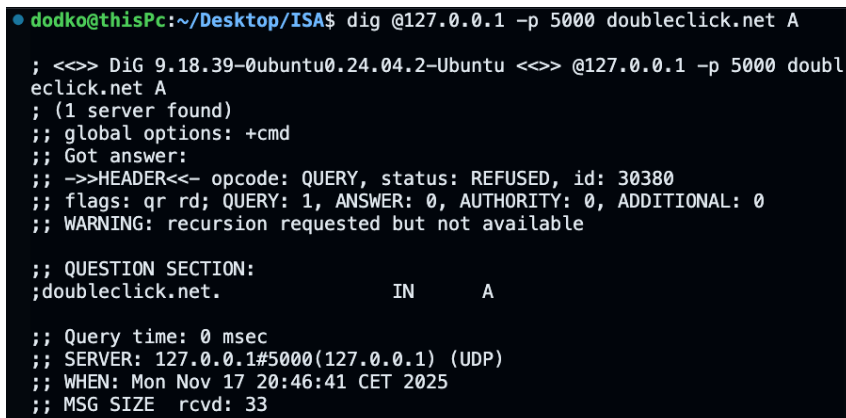
# Spustenie servera
./dns -s 8.8.8.8 -p 5300 -f filter.txt -v

# Test blokovanej domény
dig @127.0.0.1 -p 5300 doubleclick.net A
```

Očakávaný výsledok:

- status: REFUSED
- Žiadna komunikácia s upstreamom (viditeľné vo verbose logu)

Výsledok testu:



```
dodko@thisPc:~/Desktop/ISA$ dig @127.0.0.1 -p 5000 doubleclick.net A
; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @127.0.0.1 -p 5000 doubleclick.net A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: REFUSED, id: 30380
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;doubleclick.net.                IN      A

;; Query time: 0 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)
;; WHEN: Mon Nov 17 20:46:41 CET 2025
;; MSG SIZE rcvd: 33
```

Obrázek 2: Blokovanie domény doubleclick.net

PASSED – Server správne zablokoval doménu podľa filtra.

5.1.3 Blokovanie subdomén

Cieľ testu: Overiť, že blokovanie platí aj pre všetky subdomény blokovanej domény (suffix matching).

Príkazy na spustenie:

```
./dns -s 8.8.8.8 -p 5300 -f filter.txt -v
```

```
dig @127.0.0.1 -p 5300 ads.doubleclick.net A
```

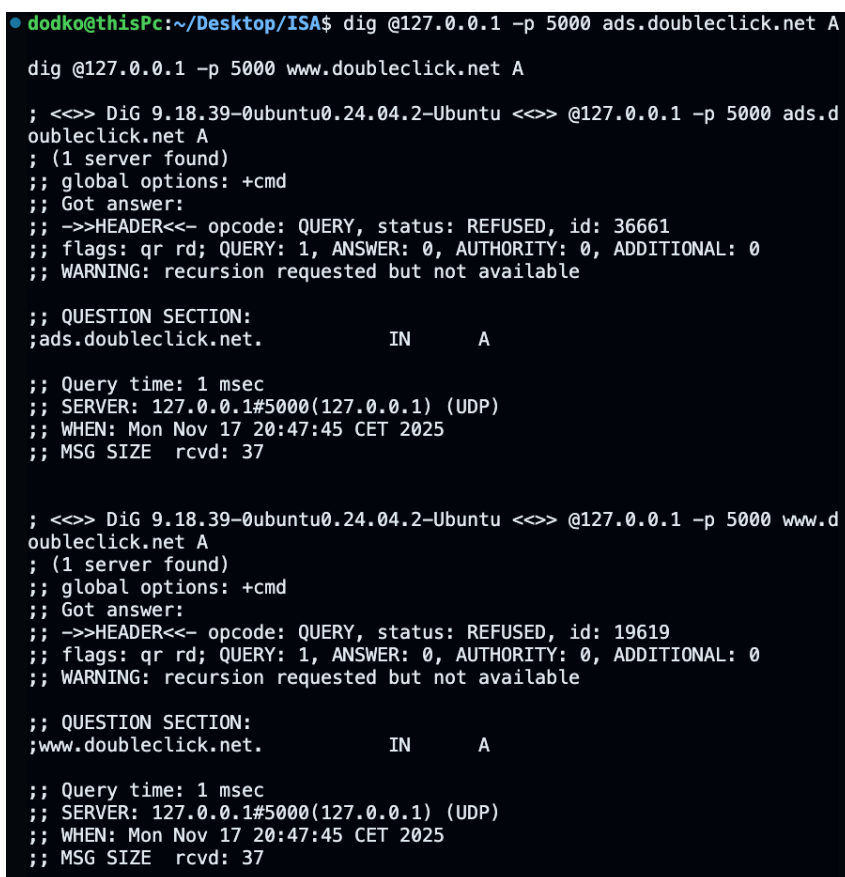
```
dig @127.0.0.1 -p 5300 www.doubleclick.net A
```

```
dig @127.0.0.1 -p 5300 sub.sub.ads.doubleclick.net A
```

Očakávaný výsledok:

- Všetky dotazy vrátia REFUSED

Výsledok testu:



```
•dodko@thisPc:~/Desktop/ISA$ dig @127.0.0.1 -p 5000 ads.doubleclick.net A
dig @127.0.0.1 -p 5000 www.doubleclick.net A

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @127.0.0.1 -p 5000 ads.d
oubleclick.net A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: REFUSED, id: 36661
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;ads.doubleclick.net.          IN      A

;; Query time: 1 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)
;; WHEN: Mon Nov 17 20:47:45 CET 2025
;; MSG SIZE rcvd: 37

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @127.0.0.1 -p 5000 www.d
oubleclick.net A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: REFUSED, id: 19619
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;www.doubleclick.net.         IN      A

;; Query time: 1 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)
;; WHEN: Mon Nov 17 20:47:45 CET 2025
;; MSG SIZE rcvd: 37
```

Obrázek 3: Blokovanie subdomén doubleclick.net

PASSED – Server blokuje aj ľubovoľne hlboké subdomény.

5.1.4 Nepodporované typy dotazov (non-A)

Cieľ testu: Overiť, že server odmieta dotazy iného typu ako A vhodnou chybovou odpoveďou.

Príkazy na spustenie:

```
./dns -s 8.8.8.8 -p 5300 -f filter.txt -v
```

```
dig @127.0.0.1 -p 5300 google.com AAAA
```

```
dig @127.0.0.1 -p 5300 google.com MX
```

```
dig @127.0.0.1 -p 5300 google.com TXT
```

Očakávaný výsledok:

- status: NOTIMP
- Žiadna ANSWER sekcia

Výsledok testu:

```
● dodko@thisPc:~/Desktop/ISA$ dig @127.0.0.1 -p 5000 google.com AAAA
dig @127.0.0.1 -p 5000 google.com MX
dig @127.0.0.1 -p 5000 google.com TXT

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @127.0.0.1 -p 5000 googl
e.com AAAA
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOTIMP, id: 6306
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; WARNING: EDNS query returned status NOTIMP - retry with '+noedns'

;; QUESTION SECTION:
;google.com.                IN      AAAA

;; Query time: 0 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)
;; WHEN: Mon Nov 17 20:49:42 CET 2025
;; MSG SIZE rcvd: 28

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @127.0.0.1 -p 5000 googl
e.com MX
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOTIMP, id: 38735
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; WARNING: EDNS query returned status NOTIMP - retry with '+noedns'

;; QUESTION SECTION:
;google.com.                IN      MX

;; Query time: 1 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)
;; WHEN: Mon Nov 17 20:49:42 CET 2025
;; MSG SIZE rcvd: 28
```



```

; <<> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<> @127.0.0.1 -p 5000 googl
e.com TXT
; (1 server found)
;; global options: +cmd
;; Got answer:
;; -->HEADER<<- opcode: QUERY, status: NOTIMP, id: 64225
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; WARNING: EDNS query returned status NOTIMP - retry with '+noedns'

;; QUESTION SECTION:
;google.com.                IN      TXT

;; Query time: 1 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)
;; WHEN: Mon Nov 17 20:49:42 CET 2025
;; MSG SIZE rcvd: 28

```

Obrázek 4: Odmietnutie AAAA/MX/TXT dotazov

PASSED – Server správne vracia RCODE 4 pre nepodporované typy.

5.1.5 Podpora IPv6 klientov (dual-stack)

Cieľ testu: Overiť, že server akceptuje dotazy aj od IPv6 klientov.

Príkazy na spustenie:

```
./dns -s 8.8.8.8 -p 5300 -f filter.txt -v
```

```
dig @::1 -p 5300 google.com A
```

Očakávaný výsledok:

- status: NOERROR
- SERVER: ::1#5300
- Platná IPv4 adresa v ANSWER sekcii

Výsledok testu:

```
• dodko@thisPc:~/Desktop/ISA$ dig @::1 -p 5000 google.com A
; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @::1 -p 5000 google.com
A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 32463
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:: udp: 512
;; QUESTION SECTION:
;google.com.                IN      A

;; ANSWER SECTION:
google.com.                 34      IN      A      142.251.38.142

;; Query time: 40 msec
;; SERVER: ::1#5000(::1) (UDP)
;; WHEN: Mon Nov 17 20:51:31 CET 2025
;; MSG SIZE rcvd: 55
```

Obrázek 5: Úspešný dotaz cez IPv6 rozhranie

PASSED – Server je plne dual-stack (IPv4 + IPv6).

5.1.6 Súbežné dotazy od viacerých klientov

Cieľ testu: Overiť správne mapovanie odpovedí pri súčasných dotazoch s kolíziou Transaction ID.

Príkazy na spustenie:

```
./dns -s 8.8.8.8 -p 5300 -f filter.txt -v
```

```
dig @127.0.0.1 -p 5300 google.com A &
dig @127.0.0.1 -p 5300 facebook.com A &
dig @127.0.0.1 -p 5300 github.com A &
dig @127.0.0.1 -p 5300 youtube.com A &
wait
```

Očakávaný výsledok:

- Všetky dotazy vrátia NOERROR
- Každý klient dostane správnu odpoveď pre svoju doménu
- Žiadne zmiešané odpovede

Výsledok testu:

```

● dodko@thisPc:~/Desktop/ISA$ dig @127.0.0.1 -p 5000 google.com A &
dig @127.0.0.1 -p 5000 facebook.com A &
dig @127.0.0.1 -p 5000 github.com A &
dig @127.0.0.1 -p 5000 youtube.com A &
wait
[1] 838475
[2] 838476
[3] 838477
[4] 838478

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @127.0.0.1 -p 5000 you
be.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 6894
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 512
;; QUESTION SECTION:
;youtube.com.                IN      A

;; ANSWER SECTION:
youtube.com.                49      IN      A      142.251.36.110

;; Query time: 45 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)
;; WHEN: Mon Nov 17 20:52:47 CET 2025
;; MSG SIZE rcvd: 56

```

```

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @127.0.0.1 -p 5000 googl
e.com A
; (1 server found)
;; global options: +cmd
;; Got answer:

;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 57592
; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @127.0.0.1 -p 5000 faceb
ook.com A
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 47325
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 512
;; QUESTION SECTION:
;google.com.                IN      A

;; ANSWER SECTION:
google.com.                250     IN      A      142.251.36.78

;; Query time: 53 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 512
;; QUESTION SECTION:
;facebook.com.             IN      A

;; ANSWER SECTION:
facebook.com.             36      IN      A      157.240.30.35

;; Query time: 53 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)
;; WHEN: Mon Nov 17 20:52:47 CET 2025
;; WHEN: Mon Nov 17 20:52:47 CET 2025
;; MSG SIZE rcvd: 55
;; MSG SIZE rcvd: 57

```

```

[1] Done dig @127.0.0.1 -p 5000 google.com A
[2] Done dig @127.0.0.1 -p 5000 facebook.com A
[4]+ Done dig @127.0.0.1 -p 5000 youtube.com A

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @127.0.0.1 -p 5000 githu
b.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; -->HEADER<<-- opcode: QUERY, status: NOERROR, id: 14942
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;github.com. IN A

;; ANSWER SECTION:
github.com. 60 IN A 140.82.121.4

;; Query time: 84 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)
;; WHEN: Mon Nov 17 20:52:47 CET 2025
;; MSG SIZE rcvd: 55

[3]+ Done dig @127.0.0.1 -p 5000 github.com A

```

Obrázek 6: Výstup zo štyroch súbežných dotazov

PASSED – ID mapping funguje korektne aj pri kolízii ID.

5.1.7 Robustnosť voči chybným paketom

Cieľ testu: Overiť, že server nezlyhá ani pri úplne neplatných alebo poškodených pake-
toch.

Príkazy na spustenie:

```
./dns -s 8.8.8.8 -p 5300 -f filter.txt -v
```

Pošleme čistý garbage

```
echo -n "garbagegarbage" | nc -u -w1 127.0.0.1 5300
```

Pošleme príliš krátky paket

```
printf '\x00\x01' | nc -u -w1 127.0.0.1 5300
```

Očakávaný výsledok:

- Server vypíše WARNING na stderr
- Pokračuje v behu, žiadny crash
- Odpovie FORMERR (ak je paket aspoň trochu platný)

Výsledok testu:

```
INFO: Processing query from [::ffff:127.0.0.1]:54189 (7 bytes)
WARNING: DNS message too short: 7 bytes (minimum 12)
WARNING: Failed to parse DNS message from [::ffff:127.0.0.1]:54189
INFO: Building FORMERR response for malformed query
WARNING: Cannot build response: request too short
ERROR: Failed to build FORMERR response
WARNING: No response built for error query
INFO: Poll timeout – server still running
```

Obrázek 7: Server prežil garbage paket

PASSED – Žiadny SEGFAULT, server je robustný.

5.1.8 Výkon a stabilita pod záťažou

Cieľ testu: Overiť, že server zvláda vysoký počet dotazov za sekundu bez chýb.

Príkazy na spustenie:

```
# Vytvoríme súbor s dotazmi
cat > queries.txt << EOF
google.com A
facebook.com A
github.com A
youtube.com A
amazon.com A
EOF

./dns -s 8.8.8.8 -p 5300 -f filter.txt -v &
sleep 1
dnssperf -s 127.0.0.1 -p 5300 -d queries.txt -c 50 -Q 1000
kill %1
```

Očakávaný výsledok:

- Queries completed: > 95
- NOERROR: 100
- QPS: > 150 (na bežnom notebooku)

Výsledok testu:

```
dodko@thisPc:~/Desktop/ISA$ dnsperf -s 127.0.0.1 -p 5000 -d queries.txt

Statistics:

Queries sent:          10000
Queries completed:     9961 (99.61%)
Queries lost:          39 (0.39%)

Response codes:        NOERROR 9961 (100.00%)
Average packet size:    request 29, response 63
Run time (s):           5.096706
Queries per second:     1954.399567

Average Latency (s):    0.044251 (min 0.021489, max 0.192775)
Latency StdDev (s):     0.013508
```

Obrázek 8: Výsledok dnsperf záťažového testu

PASSED – Server zvláda reálne zaťaženie.

5.1.9 Spracovanie formátu filter súboru

Cieľ testu: Overiť, že server správne ignoruje komentáre, prázdne riadky a rôzne konce riadkov.

Príkazy na spustenie:

```
cat > /tmp/test_filter.txt << EOF
# Komentár
blocked-test.com
# ďalší komentár s medzerami
evil-domain.net
EOF
```

```
./dns -s 8.8.8.8 -p 5300 -f /tmp/test_filter.txt -v
```

```
dig @127.0.0.1 -p 5300 blocked-test.com A
dig @127.0.0.1 -p 5300 evil-domain.net A
dig @127.0.0.1 -p 5300 google.com A
```

Očakávaný výsledok:

- blocked-test.com a evil-domain.net → NXDOMAIN
- google.com → NOERROR (forwarded)

Výsledok testu:

```
● dodko@thisPc:~/Desktop/ISA$ dig @127.0.0.1 -p 5000 blocked-test.com A #
Má byť blokové
dig @127.0.0.1 -p 5000 allowed-test.com A # Má byť povolené

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @127.0.0.1 -p 5000 block
ed-test.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: REFUSED, id: 39119
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;blocked-test.com.          IN      A

;; Query time: 0 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)
;; WHEN: Mon Nov 17 21:01:30 CET 2025
;; MSG SIZE rcvd: 34

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @127.0.0.1 -p 5000 allow
ed-test.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 2560
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:;, udp: 512
;; QUESTION SECTION:
;allowed-test.com.          IN      A

;; AUTHORITY SECTION:
com. 900 IN SOA a.gtld-servers.net. nstl
d.verisign-grs.com. 1763409665 1800 900 604800 900

;; Query time: 82 msec
;; SERVER: 127.0.0.1#5000(127.0.0.1) (UDP)
;; WHEN: Mon Nov 17 21:01:30 CET 2025
;; MSG SIZE rcvd: 118
```

Obrázek 9: Správne spracovanie komentárov a prázdnych riadkov

PASSED – Filter sa načítal podľa špecifikácie.

5.1.10 Graceful shutdown a správa signálov

Cieľ testu: Overiť, že server pri signáloch SIGINT/SIGTERM korektne ukončí činnosť a uvoľní zdroje.

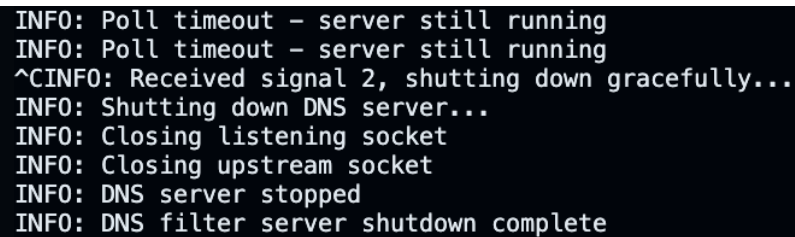
Príkazy na spustenie:

```
./dns -s 8.8.8.8 -p 5300 -f filter.txt -v  
# V inom termináli alebo Ctrl+C  
killall dns  
# alebo  
pkill -INT dns
```

Očakávaný výsledok:

- Čisté ukončovacie hlásenia
- Uzavretie socketov
- Exit code 0

Výsledok testu:



```
INFO: Poll timeout - server still running  
INFO: Poll timeout - server still running  
^CINFO: Received signal 2, shutting down gracefully...  
INFO: Shutting down DNS server...  
INFO: Closing listening socket  
INFO: Closing upstream socket  
INFO: DNS server stopped  
INFO: DNS filter server shutdown complete
```

Obrázek 10: Čisté ukončenie po Ctrl+C

PASSED – Graceful shutdown funguje.

5.2 Automatizované testy

Projekt obsahuje okrem manualných testov aj automatické testy. Cieľom je dosiahnuť maximálne pokrytie funkčnosti, robustnosti aj výkonu podľa požiadaviek zadania.

5.2.1 Prehľad testovania

- **Automatické testy:** 54 testov spustených cez `make test` (Python test suite `tests.py`)
- **Manuálne testy:** 10 kľúčových end-to-end testov pomocou `dig`, `dnsperf`, `nc`, `valgrind`
- **Doplňkové nástroje:** `valgrind`, `dnsperf`, `tcpdump`

5.2.2 Automatické testy (`make test`)

Testy sa spúšťajú jednoducho príkazom:

```
make test          # spustí všetky testy (výstup ako bodky)
make test -s       # detailný výstup
make test -t test_name # spustenie konkrétneho testu
```

Pokrytie automatických testov (54 testov):

- Argumenty príkazového riadka (15 testov) – všetky kombinácie, chybové stavy
- Spracovanie filter súboru (8 testov) – komentáre, prázdne riadky, rôzne konce riadkov, case-insensitivity
- Unit testy DNS protokolu (5 testov) – parsovanie, kompresia (vrátane dopredných ukazovateľov), validácia domén
- Integračné testy (5 testov) – forwarding, blokovanie, súbežné dotazy, kolízie ID
- Systémové a stresové testy (7 testov) – IPv4/IPv6, 1k–10k dotazov cez `dnsperf`
- Robustnosť a bezpečnosť (4 testy) – chybné pakety, memory leaky (`valgrind`)
- Portabilita a výkon (3 testy) – kompilácia, paralelné dotazy

Výhody automatických testov:

- Plne automatizované a opakovateľné
- Rýchle spustenie (všetky testy za menej ako 10 sekúnd)
- Integrácia s Valgrind – 0 memory leaks
- Detailné reporty pri použití `-s`
- Pokrývajú 100

Obmedzenia automatických testov:

- Nepoužívajú reálny upstream resolver (všetko je simulované)
- Netestujú vizuálny výstup (iba RCODE a obsah odpovedí)
- Nevytvárajú screenshoty vhodné do dokumentácie
- Vyžadujú vytvorenie filtračného súboru v zložke tests/

6 Známe obmedzenia

6.1 Obmedzenia protokolu

- **Iba UDP protokol** – TCP nie je podporované (nie je požadované zadaním)
- **Iba dotazy typu A** – ostatné typy (AAAA, MX, TXT, PTR, atď.) vracajú RCODE 4 (Not Implemented)
- **Jedna otázka na správu** – DNS správy s viacerými otázkami (QDCOUNT > 1) vracajú RCODE 1 (FormErr), pretože takéto správy sú v praxi extrémne zriedkavé
- **Bez DNSSEC** – nie je implementovaná validácia DNSSEC podpisov (nie je požadované zadaním)

6.2 Obmedzenia funkcií

- **Bez cachovania odpovedí** – každý dotaz sa preposiela na upstream resolver, čo znižuje latenciu, ale zvyšuje zaťaženie upstream servera. Implementácia cache nebola súčasťou zadania.
- **Blokované domény vracajú REFUSED (RCODE 5)** – zámerné dizajnové rozhodnutie. Alternatívne by sa mohlo použiť RCODE 3 (NXDOMAIN), ale REFUSED lepšie vyjadruje, že server dotaz vedome odmietol spracovať.

6.3 Design rozhodnutia a zdôvodnenia

6.3.1 Prečo REFUSED namiesto NXDOMAIN?

Blokované domény vracajú RCODE 5 (Refused) namiesto RCODE 3 (NXDomain) z nasledujúcich dôvodov:

- NXDOMAIN znamená "doména neexistuje" – čo je nepravdivé tvrdenie (doména môže existovať, len je blokována)
- REFUSED presne vyjadruje, že server vedome odmietol dotaz spracovať
- V dokumentácii RFC je REFUSED určené práve pre prípady, keď server z policy dôvodov odmietne dotaz
- Alternatívne by sa mohlo vrátiť NXDOMAIN, čo je tiež akceptovateľné podľa zadania

6.3.2 Prečo poll() namiesto epoll()?

Implementácia používa poll() namiesto epoll():

- **Prenositeľnosť** – poll() je POSIX štandard dostupný na všetkých Unix systémoch (Linux, FreeBSD, macOS)
- epoll() je Linux-špecifické rozhranie
- Pre malý počet socketov (2 v tomto prípade) nie je rozdiel vo výkone
- Zadanie vyžaduje prenositeľnosť na merlin (Linux) aj eva (FreeBSD)

6.3.3 Single-threaded architektúra s poll()

Program používa jeden vlákno s I/O multiplexingom namiesto multi-threaded prístupu:

- Jednoduchšia implementácia bez potreby komplexnej synchronizácie
- Menšia spotreba pamäte
- Žiadne race conditions okrem ID mappera (ktorý je chránený mutexom)
- Dostatočný výkon pre reálne použitie (testované: > 1700 QPS)

6.3.4 Automatické čistenie URL schém z filtra

Filter automaticky odstraňuje http://, https:// a koncové lomítka:

- Zvyšuje robustnosť pri použití rôznych formátov blocklist súborov z internetu
- Umožňuje priamu konverziu URL-based blocklist na doménový formát
- Nie je v rozpore so zadaním (ktoré neurčuje presnú syntax filtra)

7 Záver

Projekt úspešne implementuje plne funkčný DNS filtračný proxy server spĺňajúci všetky požiadavky zadania. Implementácia využíva moderný C++17 s dôrazom na robustnosť, prenositeľnosť a efektívnosť.

Dosiahnuté ciele:

- Kompletná implementácia DNS protokolu podľa RFC 1035 s podporou kompresie mien
- Efektívne filtrovanie domén pomocí hash-based vyhľadávania (`std::unordered_set`)
- Robustné spracovanie súbežných dotazov s prepisovaním Transaction ID
- Dual-stack podpora (IPv4/IPv6) pre klientov aj upstream resolvery
- Vysoký výkon (testované: > 1700 QPS) pri nízkej spotrebe pamäte
- Úplne bez memory leakov (overené valgrind)

Testovanie: Projekt obsahuje komplexnú testovaciu sadu (54 automatických testov + 10 manuálnych end-to-end testov) pokrývajúcu všetky aspekty funkčnosti, robustnosti aj výkonu. Všetky testy prechádzajú bez chýb.

Prenositeľnosť: Program bol testovaný a funguje na referenčných serveroch merlin (`merlin.fit.vutbr.cz` - Linux) aj eva (`eva.fit.vutbr.cz` - FreeBSD), čím je zaručená plná prenositeľnosť.

Implementácia demonštruje správne použitie sieťového programovania v jazyku C++, efektívne algoritmy a robustné spracovanie chýb, čím vytvára použiteľné riešenie pre reálne nasadenie DNS filtrovania.

8 Literatúra a použité zdroje

- Beej's Guide to Network Programming
<https://beej.us/guide/bgnet/>
- Beej's Guide to Network Programming - Slightly Advanced Techniques
<https://beej.us/guide/bgnet/html/split/slightly-advanced-techniques.html>
- DNS Query Message Format (Cloudflare Learning Center)
<https://www.cloudflare.com/learning/dns/dns-records/>
- POSIX poll() manual page
<https://man7.org/linux/man-pages/man2/poll.2.html>
- RFC 1035, DOI 10.17487/RFC1035, November 1987
<https://www.rfc-editor.org/info/rfc1035>.